

Tests fonctionnels et d'intégration d'une application React/Laravel avec Selenium IDE

Technologie utilisée

- React
- Laravel
- MySQL
- Selenium IDE
- fetch

Introduction

Dans le cadre de ce TP, nous avons réalisé des tests fonctionnels sur une application développée avec **React** pour le frontend et **Laravel** pour le backend. L'objectif était de vérifier le bon fonctionnement des principales fonctionnalités de l'application, d'identifier les anomalies éventuelles et de confirmer leur correction à l'aide de Selenium IDE.

Liens du projet

- Application déployée (front-end) : <https://mini-garage.netlify.app/>
- API déployée (back-end) : <https://mini-garage-api-o7m5.onrender.com/>
- Présentation vidéo du projet : [Lien vers la vidéo](#)

1. Préparation de l'environnement

Avant de commencer les tests, les différents services de l'application ont été vérifiés.

- Lancement du serveur Laravel avec :

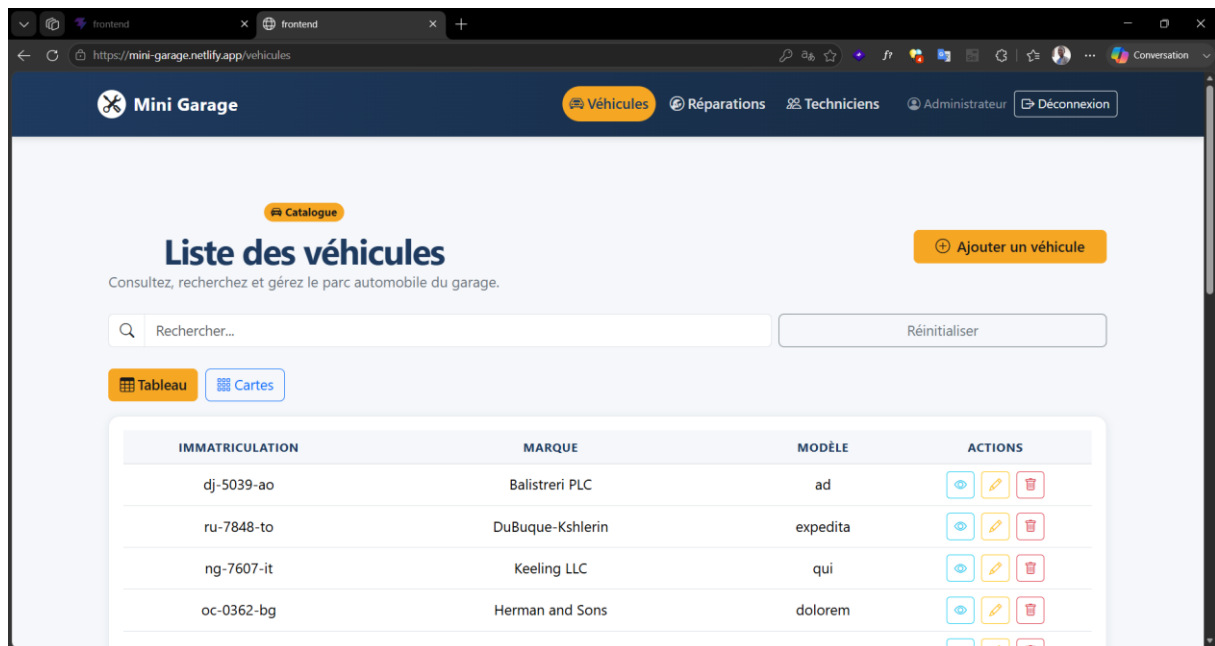
php artisan serve

- Lancement de l'application React :

npm run dev

Les différents endpoints de l'API ont également été testés afin de vérifier leur bon fonctionnement avant l'exécution des scénarios Selenium.

Capture d'écran 1 : Application React en fonctionnement

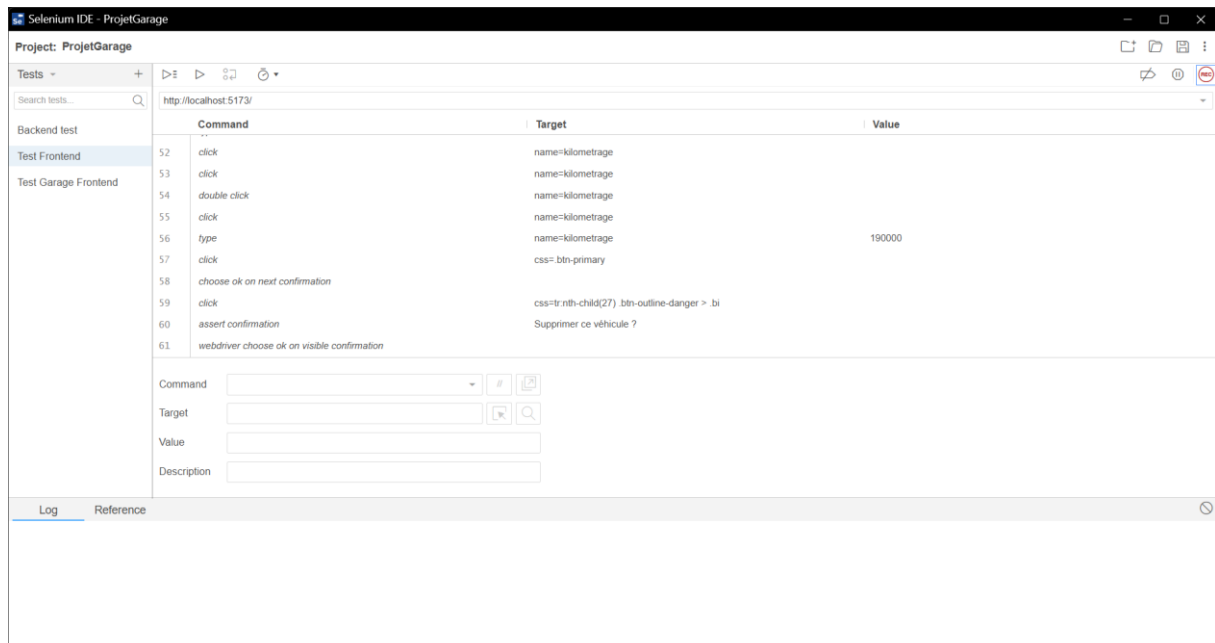


2. Installation de Selenium IDE

L'extension Selenium IDE a été installée sur un navigateur compatible.

Un nouveau projet Selenium a ensuite été créé afin d'automatiser les différents parcours utilisateurs.

Capture d'écran 2 : Projet Selenium IDE



3. Tests réalisés

Les principaux scénarios utilisateurs ont été automatisés.

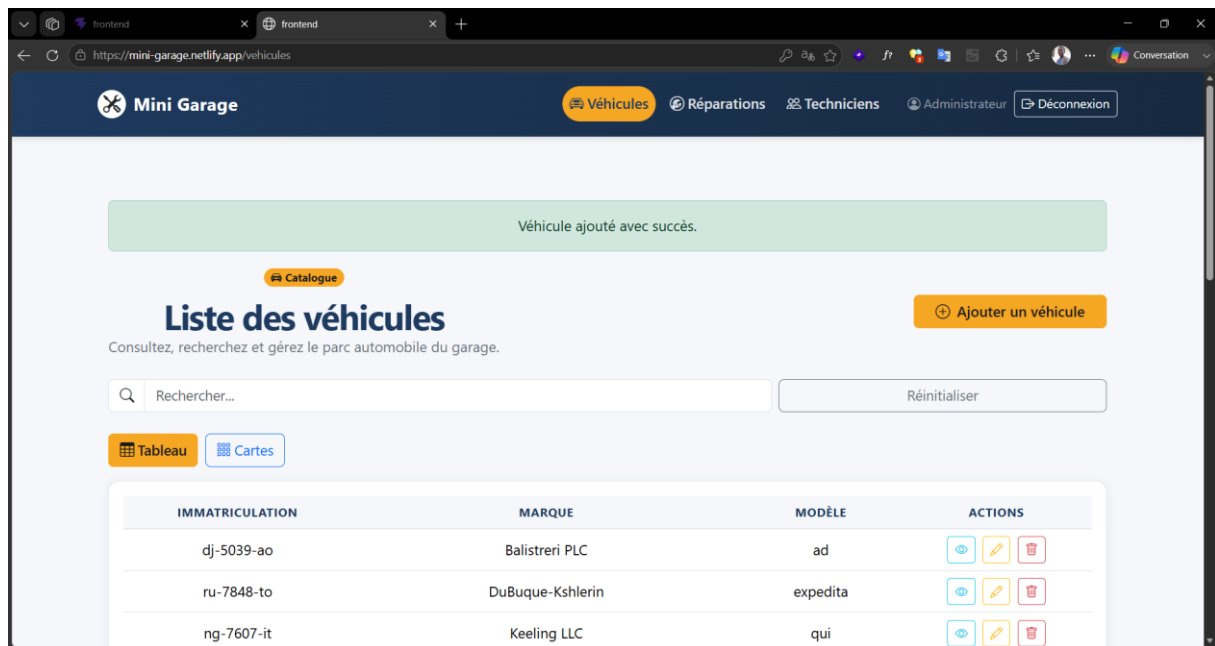
Test	Objectif	Résultat
Ajout d'un véhicule / réparation	Vérifier l'ajout d'un nouvel enregistrement	Réussi
Modification	Vérifier la mise à jour des informations	Réussi
Suppression	Vérifier la suppression d'un enregistrement	Réussi

Pour chaque scénario, des assertions ont été ajoutées afin de vérifier que le résultat attendu était bien affiché après l'exécution de l'action.

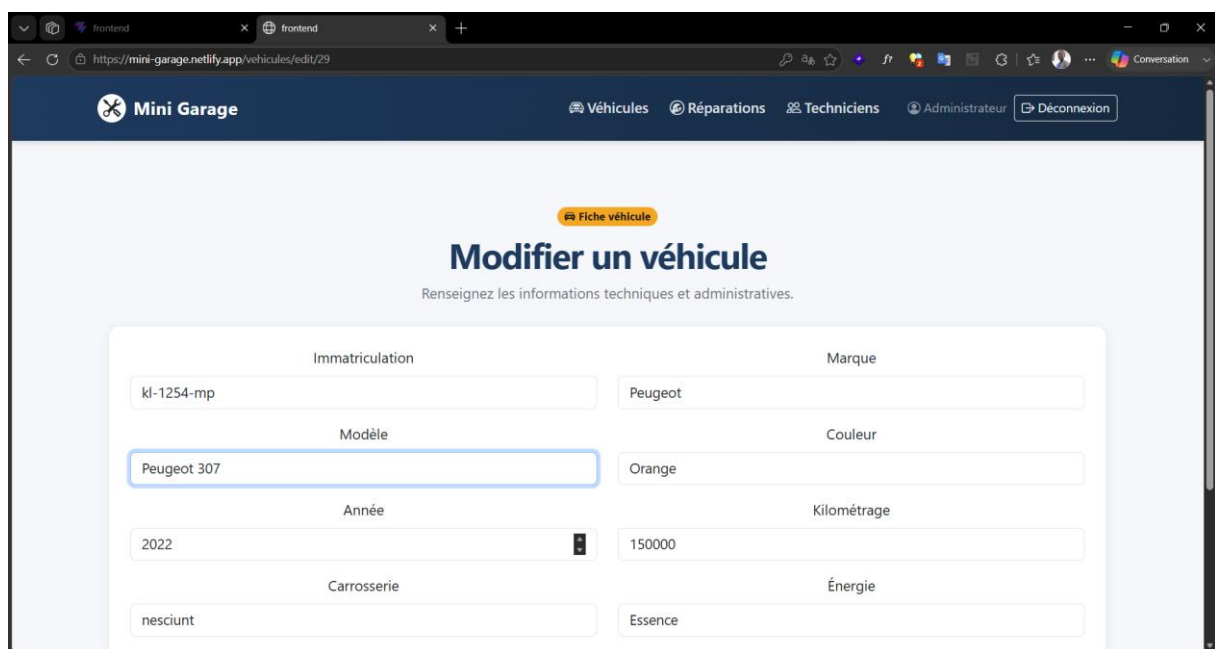
Capture d'écran 3 : Exécution du test d'ajout

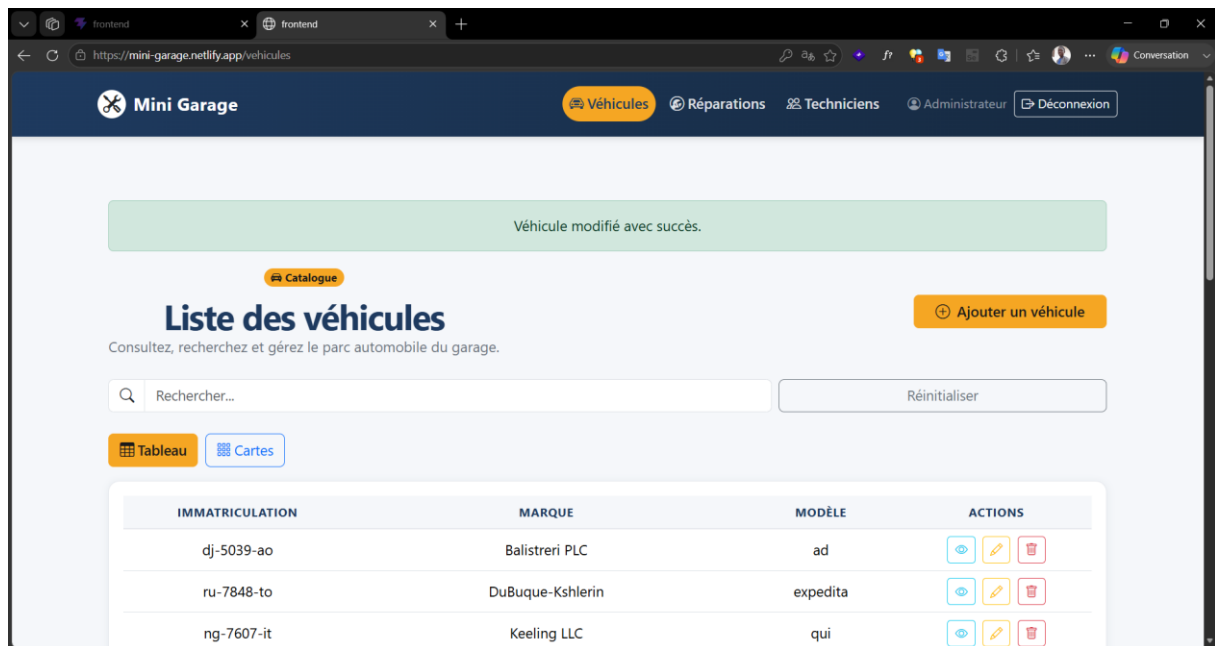
The screenshot shows a web browser window with the URL `https://mini-garage.netlify.app/vehicules/create`. The page title is "Ajouter un véhicule" with a subtitle "Renseignez les informations techniques et administratives." The form contains the following fields and values:

Field	Value
Immatriculation	kl-1254-mp
Marque	Peugeot
Modèle	Peugeot 208
Couleur	Orange
Année	2022
Kilométrage	150000
Carrosserie	nesciunt
Énergie	Essence

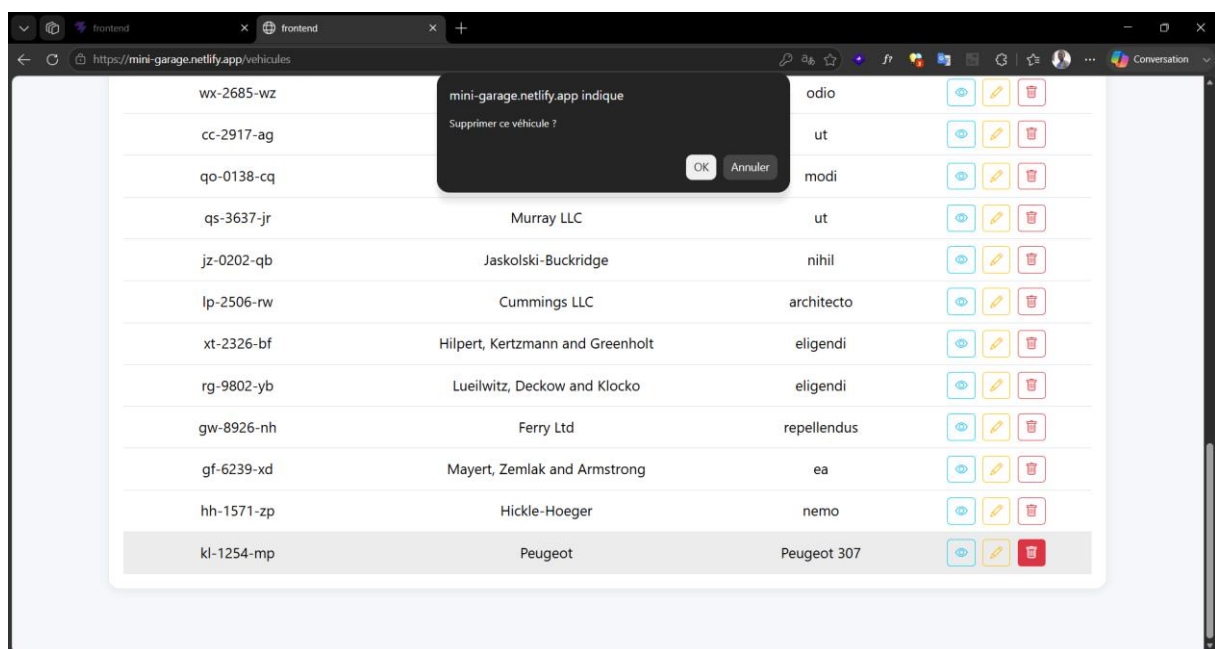


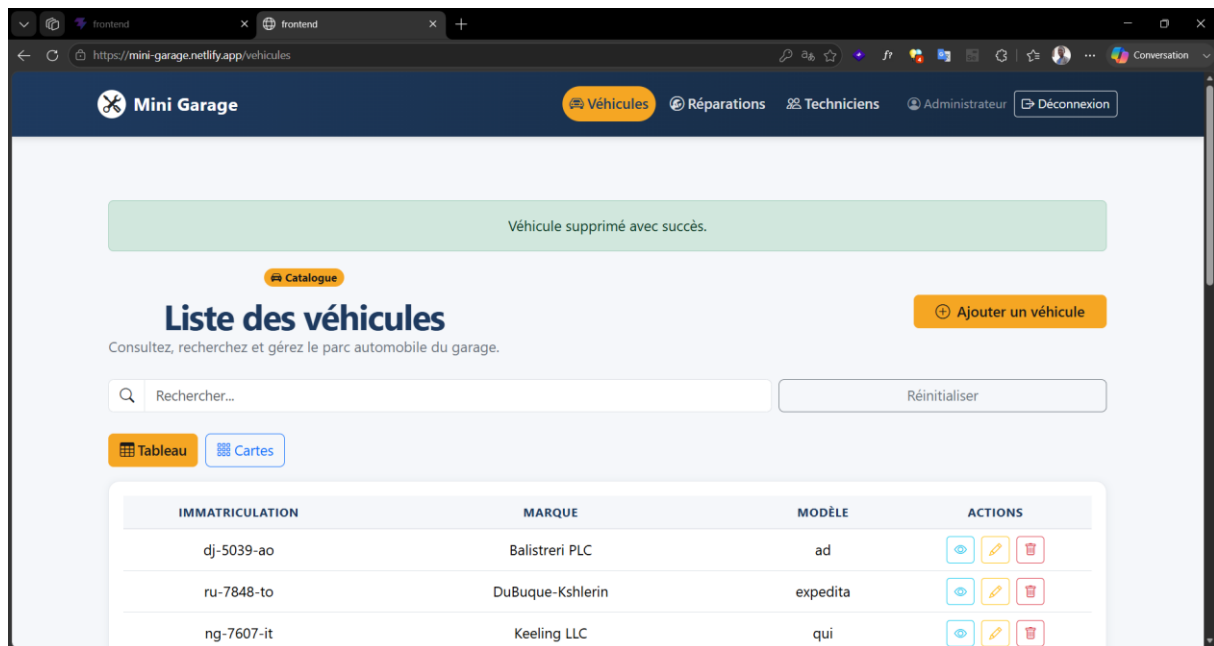
Capture d'écran 4 : Exécution du test de modification





Capture d'écran 5 : Exécution du test de suppression





4. Bug détecté

Pendant les tests, un problème a été identifié lors de l'ajout d'un véhicule.

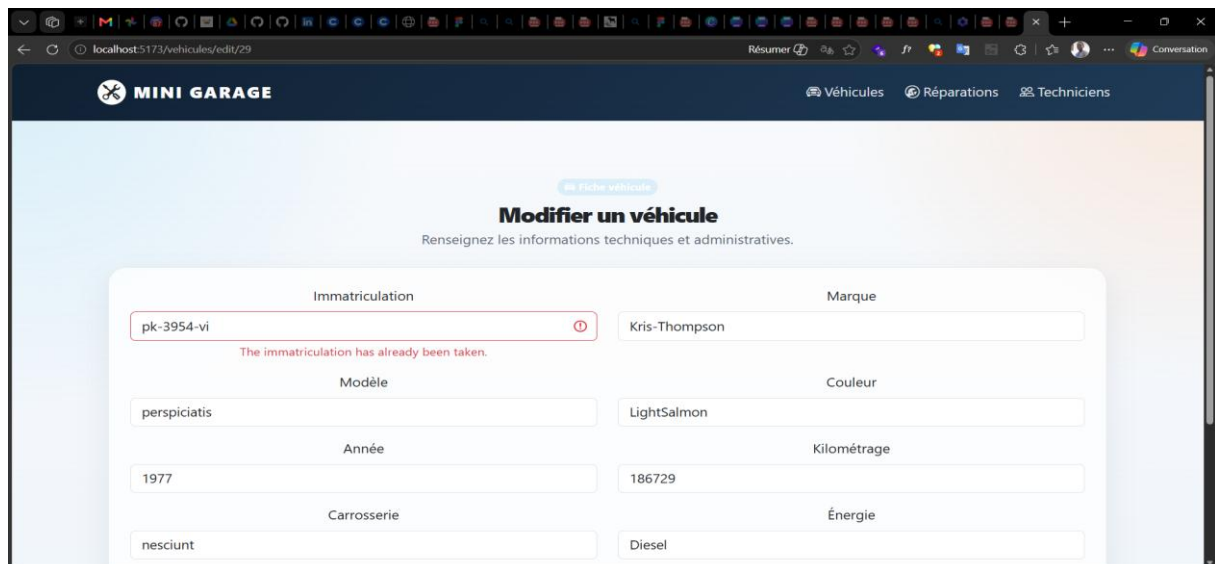
Lorsque certaines données étaient invalides ou qu'une erreur était renvoyée par le serveur, **le message d'erreur n'était pas affiché dans l'interface**. L'utilisateur ne pouvait donc pas comprendre pourquoi l'opération avait échoué.

Après analyse, le problème provenait de la gestion des erreurs côté frontend.

La logique d'affichage des erreurs a été corrigée afin que les messages soient correctement affichés à l'utilisateur.

Les tests Selenium ont ensuite été rejoués afin de confirmer que le problème avait été résolu.

Capture d'écran 6 : Erreur observée et correction



5. Tests d'intégration

Les tests réalisés ont permis de vérifier la communication entre React et Laravel.

Les opérations suivantes ont été validées :

- Création d'un enregistrement depuis React
- Enregistrement correct dans la base de données via Laravel
- Modification des données
- Suppression des données
- Actualisation correcte de l'interface utilisateur

Ces tests confirment que les échanges entre le frontend et le backend fonctionnent correctement.

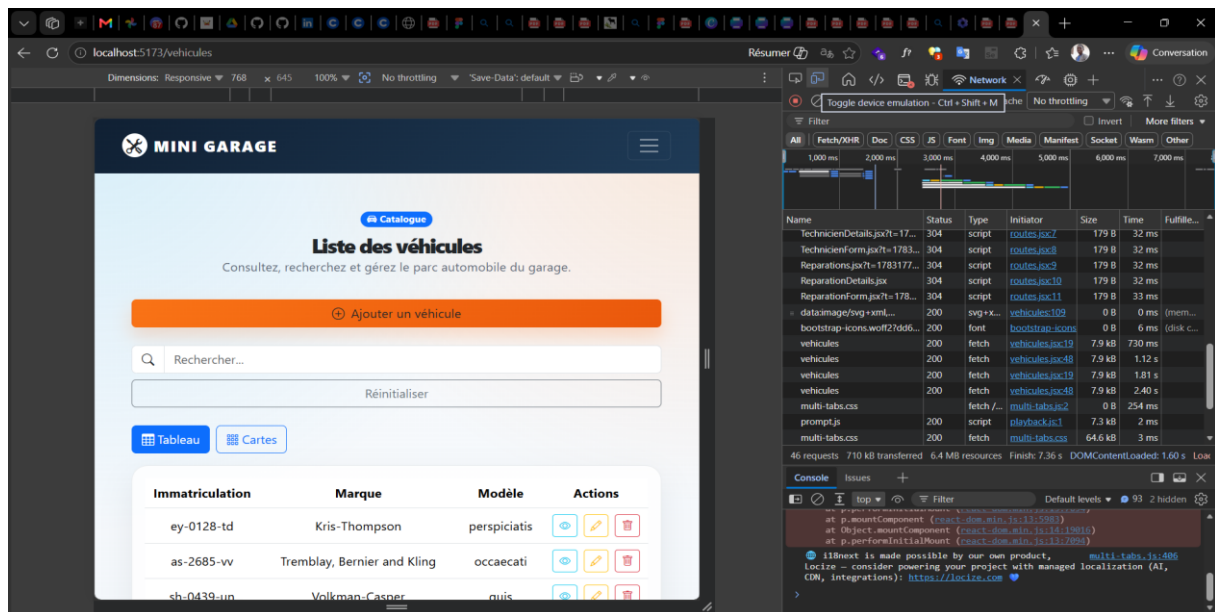
6. Observation des performances

Une observation simple des performances a été réalisée à l'aide des outils de développement du navigateur.

Les résultats montrent que :

- Le chargement de l'application est rapide en environnement local
- Les appels API sont correctement exécutés
- Aucune lenteur importante n'a été observée lors des opérations de création, modification ou suppression

Capture d'écran 7 : Onglet Network des outils de développement



7. Tests Backend (Laravel / PHPUnit)

En complément des tests fonctionnels réalisés côté interface avec Selenium IDE, des tests automatisés ont été écrits côté back-end avec PHPUnit afin de vérifier directement le comportement de l'API Laravel (validations, relations, règles métier), indépendamment de l'interface React.

Ces tests utilisent une base de données SQLite en mémoire, dédiée à l'environnement de test, afin de ne jamais impacter la base MySQL de développement.

Résultat de l'exécution (php artisan test) :

Suite de tests	Nombre de tests	Résultat
AuthApiTest	4	Réussi
VehiculeApiTest	12	Réussi
TechnicienApiTest	7	Réussi
ReparationApiTest	8	Réussi
Total	31 tests (97 assertions)	Tous réussis

Ces tests couvrent notamment :

- Le listage et la recherche de véhicules par marque ou immatriculation.
- Connexion et déconnexion de l'administrateur.
- La création, la modification et la suppression des véhicules, techniciens et réparations.
- Les règles de validation côté serveur : champs obligatoires, unicité de l'immatriculation, existence du véhicule et des techniciens liés, format de date.
- La suppression en cascade des réparations lors de la suppression d'un véhicule, et la resynchronisation des techniciens assignés lors de la modification d'une réparation.

8. Vérifications de Sécurité

En plus des tests fonctionnels, les points de sécurité suivants ont été vérifiés sur l'API :

- Validation systématique des données côté serveur avant tout enregistrement (champs requis, types, longueurs, unicité de l'immatriculation).
- Protection contre les injections SQL assurée nativement par l'ORM Eloquent (aucune requête SQL brute n'est utilisée dans l'application).
- Vérification de l'existence des identifiants liés (véhicule, techniciens) avant association à une réparation, ce qui évite les incohérences de données.
- Suppression en cascade contrôlée au niveau des migrations (clé étrangère `onDelete('cascade')`), garantissant qu'aucune réparation ne reste orpheline.

Limite identifiée : l'API ne met pas encore en place d'authentification. Cette version du projet est prévue pour un usage interne du personnel du garage ; l'ajout d'une authentification (Laravel Sanctum, déjà présent en dépendance) est identifié comme une amélioration possible pour une future itération.

9. Performance et Éco-responsabilité

En complément de l'observation réalisée dans l'onglet Network du navigateur (section 6), les points suivants ont été appliqués pour limiter la consommation de ressources :

- Chargement des relations en une seule requête côté API (Eloquent `with()`) pour éviter les requêtes N+1 lors de l'affichage des véhicules, réparations et techniciens.
- Recherche de véhicules effectuée côté serveur plutôt que de charger l'ensemble des véhicules côté client.
- Ajout d'un `debounce` (délai de 300 ms) sur le champ de recherche des véhicules, afin d'éviter un appel API à chaque frappe clavier.
- Interface sobre (Bootstrap), sans images lourdes ni ressources superflues.

Conclusion

Les différents tests réalisés avec Selenium IDE ont permis de vérifier le bon fonctionnement des principales fonctionnalités de l'application.

Trois scénarios ont été automatisés :

- Ajout

- Modification
- Suppression

Les tests ont également permis d'identifier un défaut d'affichage des messages d'erreur lors de l'ajout d'un véhicule. Après correction, les scénarios ont été rejoués avec succès, confirmant le bon fonctionnement de l'application.

Cette démarche montre l'intérêt des tests automatisés pour détecter rapidement les anomalies et garantir la qualité d'une application web. Combinés aux 31 tests automatisés PHPUnit côté back-end (section 7), ces tests couvrent désormais à la fois l'interface utilisateur et l'API, offrant une bonne couverture globale de la qualité de l'application Mini Garage.